

AI Harnesses for Production Agent Systems

Executive summary

An **AI harness** is best understood as the runtime and control layer that turns a model into a usable agent: it owns session and context management, tool and environment access, approvals, persistence, tracing, evaluation, and deployment. That definition is explicit in Microsoft’s description of agent harnesses, Anthropic’s Managed Agents documentation, AWS’s AgentCore harness documentation, and OpenAI’s Codex harness/App Server architecture. ¹

For **open-source harnesses**, the strongest current choices are not interchangeable. **LangGraph** is the best general-purpose foundation when you need explicit state, resumability, and human-in-the-loop control; **OpenAI Agents SDK** is a compact, pragmatic default for tool-using agents with tracing and guardrails; **Microsoft Agent Framework** is a strong enterprise-grade choice for Python/.NET teams that want graph workflows, OpenTelemetry, and A2A/MCP interoperability; **Google ADK** is a strong code-first option for multi-agent systems and managed deployment into Google’s stack; **LlamaIndex** and **Haystack** are especially useful when retrieval, knowledge integration, and document/data pipelines dominate; and **DSPy** stands out when you can define a metric and want to optimize program quality rather than hand-tune prompts. By public GitHub popularity, the broad open-source leaders are LangChain, AutoGen, CrewAI, LlamaIndex, DSPy, and LangGraph. ²

There is **no honest single “highest-performing harness”** across all workloads. The clearest empirical findings point the other way: framework brand matters less than architecture, tooling, evaluation discipline, and model choice. Anthropic’s own guidance says teams usually do best with **simple, composable workflows** before escalating to elaborate agent architectures; Google’s agent-systems study found that **centralized multi-agent topologies help on parallelizable tasks, decentralized topologies help on dynamic navigation**, and **multi-agent variants can hurt sequential reasoning**, sometimes by 39–70%; meanwhile DSPy’s MIPRO work is one of the few primary-source results that attributes measurable gains to the harness/programming layer itself, reporting improvements of up to **13% accuracy** on five of seven multi-stage LM programs. ³

For a **new harness** and especially for a **harness-creator tool**, the strongest design direction is a **meta-harness**: a system that can compile the same high-level specification into stateful graphs, event-driven workflows, or managed runtime configs. The recurring primitives across the best systems are explicit state, typed tools, approvals, compaction, checkpointing, streaming events, OpenTelemetry traces, and first-class eval datasets. Managed products from OpenAI, Anthropic, AWS, Microsoft, and Google all converge on the same point: shell/filesystem/browser/code execution, long-running sessions, and observability are not optional extras; they are core harness responsibilities. ⁴

Definition and market segmentation

The market is easiest to understand if “harness” is treated as a **layer**, not a brand. In current primary sources, harnesses fall into three overlapping families.

The first family is the **open-source runtime harness**: frameworks such as LangGraph, AutoGen, LlamaIndex, Haystack, CrewAI, OpenAI Agents SDK, Microsoft Agent Framework, Google ADK, and DSPy. These frameworks give developers direct control over orchestration, tools, memory, validation, testing, and deployment targets. Their value is flexibility and portability; their downside is that teams must still design and operate policy, storage, observability, and production controls themselves. ⁵

The second family is the **managed harness platform**: OpenAI’s Responses/Agent Builder/Codex surfaces, Anthropic’s Claude Managed Agents, Amazon Bedrock Agents with AgentCore Harness, Azure Foundry Agent Service, and Google’s Gemini Enterprise Agent Platform with Agent Engine. These systems reduce infrastructure work by packaging the agent loop, tool runtime, session state, managed environments, and observability into a hosted control plane. Their value is operating speed and production convenience; their downside is higher vendor coupling and less freedom to customize the deepest runtime semantics. ⁶

The third family is the **evaluation harness**. HELM provides broad, multi-metric, standardized model evaluation; EleutherAI’s `lm-evaluation-harness` remains the most established open-source benchmark harness for many few-shot model tasks; Ragas specializes in RAG evaluation; and modern runtime platforms increasingly ship native eval surfaces, including OpenAI Evals and trace grading, Azure Foundry evaluators, ADK golden datasets, Haystack evaluation, LlamaIndex evaluation, CrewAI testing, and DSPy’s `Evaluate`. For a new harness, this means evaluation should be a **first-class subsystem**, not a bolted-on dashboard. ⁷

```
xychart-beta
  title "Selected open-source harness popularity by GitHub stars"
  x-axis
  ["LangChain","AutoGen","CrewAI","LlamaIndex","DSPy","LangGraph","Semantic
  Kernel","OpenAI Agents","Haystack","ADK"]
  y-axis "GitHub stars" 0 --> 140000
  bar [135000,57600,50400,49100,34100,30900,27800,25600,25000,19400]
```

Selected stars above come from official GitHub repository pages captured in late April 2026; the Microsoft line is in transition, with **Microsoft Agent Framework** succeeding **Semantic Kernel**, so Microsoft’s ecosystem popularity is split across the legacy and successor repos. LangChain and LangGraph are also part of the same ecosystem, so their stars should not be double-counted as independent market share. ⁸

Comparative landscape

The comparison below is most useful if read as a **fit matrix**, not a leaderboard. Popularity is easiest to compare for open-source projects via GitHub stars; for closed platforms, public popularity is often only visible indirectly through ecosystem metrics, public download counts, or linked open-source repos.

Name	Repo or vendor	Primary language	Core features	License or cost	Popularity metric	Benchmarked performance where available
LangChain	<code>langchain-ai/langchain</code>	Python, JS/TS	High-level framework for agents and LLM apps; broad integrations; common entry point into LangGraph-backed execution ⁹	MIT ⁹	135k★ GitHub ⁹	No neutral public framework benchmark surfaced in primary sources reviewed
LangGraph	<code>langchain-ai/langgraph</code>	Python, JS/TS	Low-level stateful graph orchestration, durable execution, checkpointing, HITL, streaming ¹⁰	MIT ¹¹	30.9k★ GitHub ¹²	LangChain's modified τ -bench benchmark reported supervisor-architecture gains of about 50%; useful signal, but not a neutral cross-framework comparison ¹³
AutoGen	<code>microsoft/autogen</code>	Python, .NET	Multi-agent conversations, customizable interaction patterns, tools, code execution, GraphFlow-era orchestration ¹⁴	MIT code + CC-BY docs/signaling on repo page ¹⁵	57.6k★ GitHub ¹⁶	Paper reports better results than single-LLM methods across pilot applications, but not a single standardized benchmark number ¹⁷

Name	Repo or vendor	Primary language	Core features	License or cost	Popularity metric	Benchmarked performance where available
LlamaIndex	<code>run-llama/llama_index</code>	Python	Data-centric/ RAG framework, agents, agent workflows, event-driven workflows, OTel integration, deployment guides ¹⁸	MIT ¹⁹	49.1k★ GitHub ²⁰	No neutral public framework benchmark surfaced in primary sources reviewed
Haystack	<code>deepset-ai/haystack</code>	Python	Modular components and pipelines, routers, agents, evaluation, tracing, explicit control over retrieval/ memory/ generation ²¹	Apache-2.0 ²²	25k★ GitHub ²²	No neutral public framework benchmark surfaced in primary sources reviewed
CrewAI	<code>crewAIInc/crewAI</code>	Python	Multi-agent crews plus structured event-driven flows, guardrails, memory, knowledge, observability, commercial AMP extension ²³	Open-source framework; commercial AMP available ²⁴	50.4k★ GitHub; README claims 100k+ certified developers ²⁵	No neutral public framework benchmark surfaced in primary sources reviewed

Name	Repo or vendor	Primary language	Core features	License or cost	Popularity metric	Benchmarked performance where available
DSPy	<code>stanfordnlp/dspy</code>	Python	Declarative LM programs, program optimization, assertions, evaluation, ReAct/RAG/agent modules ²⁶	MIT ²⁷	34.1k★ GitHub ²⁸	MIPRO paper reports up to +13% accuracy on five of seven multi-stage LM programs using Llama-3-8B ²⁹
OpenAI Agents SDK	<code>openai/openai-agents-python</code>	Python, JS/TS	Agents, tools, handoffs, guardrails, human review, sessions, tracing, realtime agents, sandbox agents ³⁰	MIT ³¹	25.6k★ GitHub (Python SDK) ³²	No standardized public framework benchmark cited in official docs reviewed
Microsoft Agent Framework	<code>microsoft/agent-framework</code>	Python, .NET	Graph-based workflows, checkpointing, time travel, OTel, middleware, A2A/MCP, multiple providers, hosting paths ³³	MIT ³⁴	10k★ GitHub; successor to Semantic Kernel's 27.8k★ legacy repo ³⁵	Microsoft's migration guide claims better performance and memory behavior than the Semantic Kernel agent layer, but no neutral public benchmark was found ³⁶

Name	Repo or vendor	Primary language	Core features	License or cost	Popularity metric	Benchmarked performance where available
Google ADK	<code>google/adk-python</code>	Python, plus Java/Go/TS ecosystem	Code-first, model/deployment-agnostic, tools, tool confirmation, multi-agent hierarchies, Cloud Run and Agent Engine deployment, evaluation workflows ³⁷	Apache-2.0 ³⁸	19.4k★ GitHub; Google says ADK exceeded 7M downloads ³⁹	No neutral public framework benchmark surfaced in primary sources reviewed

Name	Vendor	Primary language or surface	Core features	License or cost	Popularity metric	Benchmarked performance where available
OpenAI Responses API + Agent Builder + Codex harness	OpenAI	API, Python, JS/TS	Recommended new primitive for agent-like apps; built-in tools; background mode; tracing; evals; hosted visual Agent Builder; Codex harness with bidirectional JSON-RPC App Server and cloud sandboxes ⁴⁰	Usage-based token + tool/container pricing ⁴¹	Open-source ecosystem proxy: Agents SDK 25.6k★ ³²	No neutral public framework benchmark cited in official docs reviewed

Name	Vendor	Primary language or surface	Core features	License or cost	Popularity metric	Benchmarked performance where available
Claude Managed Agents	Anthropic	API, Python SDK	Fully managed autonomous-agent harness; sessions with persistent event history; secure file/command/web/code execution; prompt caching and compaction built in ⁴²	Tokens + session runtime; official docs show \$0.08 per session-hour in running state ⁴³	Public adoption metric not disclosed in reviewed primary sources	No neutral public framework benchmark cited in official docs reviewed
Amazon Bedrock Agents + AgentCore Harness	AWS	API, CLI, managed runtime	Bedrock Agents for reasoning over APIs/data; AgentCore for secure scalable operation with runtime, memory, gateway, observability; harness config can stand up agents with minimal plumbing ⁴⁴	Usage-based: model charges plus AgentCore runtime/gateway/memory style charges ⁴⁵	Public product adoption metric not disclosed; ecosystem docs/tooling are active ⁴⁶	No neutral public framework benchmark cited in official docs reviewed

Name	Vendor	Primary language or surface	Core features	License or cost	Popularity metric	Benchmarked performance where available
Foundry Agent Service	Microsoft Azure	API, SDKs, portal	Secure design/ deploy/scale of agents; tracing to Azure Monitor/ Application Insights via OTel; built-in evaluators; tools such as Code Interpreter and Bing grounding ⁴⁷	Pay-as-you-go; tool-specific charges apply ⁴⁸	Ecosystem proxy: MAF 10k★, Semantic Kernel legacy 27.8k★ ³⁵	No neutral public framework benchmark cited in official docs reviewed
Gemini Enterprise Agent Platform	Google Cloud	API, SDKs, managed runtime	Full-stack agent platform; ADK for build/ orchestration; Agent Engine for managed serverless runtime; agent identity, governance, RAG/search integration, quotas and billing by product component ⁴⁹	Usage-based pricing by product/ component ⁵⁰	ADK proxy: 19.4k★ and 7M+ downloads ³⁹	No neutral public framework benchmark cited in official docs reviewed

Name	Vendor	Primary language or surface	Core features	License or cost	Popularity metric	Benchmarked performance where available
LangSmith Deployment	LangChain	SaaS, cloud, hybrid/ self-hosted enterprise tiers	Framework-agnostic deployment runtime for agents; observability/ evals tightly integrated; long-running workflow deployment, auth, scaling, MCP exposure ⁵¹	Requires Plus plan or above; seat + usage model on pricing page ⁵¹	Ecosystem proxy: LangGraph 30.9k★ ¹²	No neutral public framework benchmark cited in official docs reviewed
CrewAI AMP Suite	CrewAI	SaaS / on-prem	Control plane, tracing/ observability, analytics, security, cloud or on-prem deployment for CrewAI systems ⁵²	Sales-led / request demo; cloud or self-hosted factory options ⁵³	Ecosystem proxy: CrewAI 50.4k★ and 100k+ certified developers claimed ²⁵	No neutral public framework benchmark cited in official docs reviewed

A useful practical takeaway from the table is that **runtime harnesses** and **evaluation harnesses** should be chosen separately. HELM is best when breadth, standardization, and public transparency matter; `lm-evaluation-harness` remains the most reusable open benchmark runner; Ragas is strong for retrieval-centric products; OpenAI, Foundry, ADK, Haystack, LlamaIndex, CrewAI, and DSPy all offer framework-native evaluation surfaces that should be integrated into any creator tool. ⁵⁴

Architectural patterns and tradeoffs

The strongest harnesses converge on a few architectural patterns.

Stateful graph runtimes are the best fit for long-running agents, resumability, and human intervention. LangGraph exposes durable execution with configurable durability modes (`exit`, `async`, `sync`), while Microsoft Agent Framework explicitly centers graph workflows with checkpointing, streaming, time travel, and OpenTelemetry. This pattern is the clearest answer when the problem includes interruptions, approvals, or recoverable multi-step work. The tradeoff is complexity: teams must reason about state schemas, checkpoints, and replay semantics rather than just prompt chains. ⁵⁵

Event-driven workflow engines are the next major pattern. LlamaIndex workflows are explicitly event-driven and async-first; CrewAI Flows are explicitly structured, event-driven workflows that can compose more autonomous crews; ADK similarly frames agent systems as modular, code-first workflows that can scale from simple tasks to hierarchies of specialist agents. This pattern is often the best middle ground between “a chain of calls” and “a free-form autonomous agent.” It tends to be easier to test than unrestricted agent loops and easier to parallelize than pure conversation-centric orchestration. ⁵⁶

Pipeline and component architectures are especially strong for retrieval-heavy systems. Haystack’s core design is components connected by pipelines, with routers, retrievers, generators, and tools as swappable units; this makes it unusually transparent for RAG, multimodal search, and hybrid deterministic-plus-agentic systems. If the main engineering challenge is retrieval quality, document routing, ranking, or explainable knowledge flow, pipeline-first harnesses are often more useful than agent-first harnesses. ⁵⁷

Declarative optimized programs are the main differentiator of DSPy. DSPy’s wager is that prompt strings are the wrong abstraction, and that developers should instead express modular LM programs with metrics, optimizers, and assertions. When a high-quality evaluation set exists, this is the most evidence-backed route to squeezing accuracy gains out of the harness layer itself. The tradeoff is that DSPy is less of a turnkey operating harness than LangGraph, MAF, or managed platforms; it shines most when paired with strong evaluation discipline and explicit task metrics. ⁵⁸

Managed harnesses are increasingly a distinct category. Anthropic’s Managed Agents, AWS’s AgentCore harness, OpenAI’s Codex harness and App Server, and Microsoft’s agent-harness patterns all formalize the same boundary: the model “brain” should not be tightly entangled with tool execution, shells, filesystems, approvals, or session infrastructure. That layer should be explicit, inspectable, and policy-controlled. In practice, this is one of the most important design lessons for a new harness or creator tool. ⁵⁹

```
flowchart LR
    U[User or calling app] --> G[API gateway]
    G --> P[Policy and approval engine]
    P --> O[Orchestrator]
    O --> S[Session and checkpoint store]
    O --> M[Model router]
    O --> T[Tool gateway]
    O --> R[Retriever and memory services]
    T --> MCP[MCP servers]
    T --> HTTP[HTTP or function tools]
    T --> ENV[Sandbox or shell environment]
    O --> V[Verifier or evaluator]
    O --> L[Telemetry and traces]
    V --> L
    L --> A[Artifacts and outputs]
```

This flow combines patterns that recur across OpenAI Agents/Responses, LangGraph, MAF, ADK, Haystack, Claude Managed Agents, and AgentCore: explicit policy checks, state reload/checkpoint, model planning, tool execution, retrieval, verification, and structured telemetry. ⁶⁰

The most important **performance** lesson is not “use more agents.” Anthropic’s guidance argues for starting with simple composable workflows, and Google’s controlled scaling study reached a similar conclusion: task structure should determine topology. Centralized multi-agent coordination helped on highly parallelizable finance-style reasoning, decentralized patterns helped on dynamic browsing/navigation, and multi-agent systems degraded sequential reasoning tasks. In other words, harness design should make **single-agent, workflow, centralized multi-agent, and decentralized multi-agent** all available, but none should be the default everywhere. ⁶¹

The biggest **reliability** tradeoff is between latency and durability. Synchronous checkpointing, managed session history, background execution, and context compaction improve recoverability and long-run correctness, but they add storage writes, orchestration overhead, and operational cost. LangGraph documents this directly in its durability modes; Microsoft’s harness guidance emphasizes compaction for long histories; OpenAI’s background mode exists specifically for long-running tasks; Anthropic surfaces prompt caching and session runtime explicitly in pricing; and managed platforms from AWS and Google make runtime, memory, and observability first-class billing or product surfaces. ⁶²

The largest **security** tradeoff is between capability and trust surface. Local shell tools, browser tools, code execution, MCP servers, and external search materially increase attack and compliance surface. Primary-source guidance is clear here: Microsoft recommends isolation and explicit approvals for local shell execution; OpenAI distinguishes hosted and local/private MCP and emphasizes approvals and guardrails; Azure documents that Bing grounding sends data outside Azure’s usual compliance boundary; Google emphasizes agent identity and audit trails; and AWS emphasizes secure gateways and observability around tool access. A new harness should therefore treat policy and approval as first-class middleware, not optional wrappers. ⁶³

The largest **cost** tradeoff is between self-hosted flexibility and managed convenience. Open-source harnesses minimize licensing cost but push runtime, storage, and observability burden onto the team. Managed harnesses shift engineering effort into direct usage charges: OpenAI bills tokens plus tool/container costs; Anthropic bills tokens plus session runtime; Azure adds tool-specific charges such as Code Interpreter and Bing grounding; AWS bills Bedrock and AgentCore resources; Google charges by Agent Platform product components. Teams designing a creator tool should therefore model **cost per successful run**, not just token cost. ⁶⁴

Reference architecture for a harness-creator tool

The right product architecture is **not** “yet another hardcoded agent loop.” It should be a **compiler and control plane** that accepts a high-level harness spec and emits one of several runtime forms: a graph runtime, an event-driven workflow, a pipeline DAG, or a managed-runtime bundle. That recommendation is grounded in the diversity of successful patterns: graphs in LangGraph and MAF, event-driven workflows in LlamaIndex/CrewAI/ADK, component pipelines in Haystack, and managed harnesses from OpenAI, Anthropic, and AWS. ⁶⁵

```
flowchart TB
    Studio[Creator studio and visual editor] --> Compiler[Spec compiler and validator]
    Compiler --> IR[Canonical intermediate representation]
```

```

IR --> GraphTarget[Graph runtime target]
IR --> WorkflowTarget[Event workflow target]
IR --> PipelineTarget[Pipeline target]
IR --> ManagedTarget[Managed runtime package]

GraphTarget --> Runtime[Orchestrator runtime]
WorkflowTarget --> Runtime
PipelineTarget --> Runtime
ManagedTarget --> Runtime

Runtime --> Model[Model adapters]
Runtime --> Tools[Tool and MCP gateway]
Runtime --> Memory[Session, memory, checkpoint store]
Runtime --> Policy[Guardrails and approvals]
Runtime --> Eval[Evals and trace grading]
Runtime --> Deploy[Deployment controller]
Runtime --> OTel[OpenTelemetry exporter]

```

A practical module breakdown for the creator tool is shown below.

Module	Responsibility
Spec registry	Store harness templates, versions, migration metadata, ownership, and environment overlays
Compiler	Validate template schemas, expand macros, infer runtime graph, and emit deployment bundles
IR model	Canonical representation of agents, steps, events, tools, policies, memory, and eval hooks
Runtime orchestrator	Execute runs, stream events, checkpoint state, and resume after interruption
Model adapter layer	Normalize vendor/model calls, structured output, retries, routing, token accounting
Tool gateway	Execute local tools, hosted tools, HTTP functions, and MCP servers behind one typed contract
Policy engine	Apply approvals, allowlists, PII filtering, side-effect classification, output validation
Session and checkpoint store	Persist short-term state, resumable step state, working memory, branch histories
Artifact store	Persist files, screenshots, patches, reports, execution summaries, evaluation artifacts

Module	Responsibility
Telemetry and eval service	Emit traces, metrics, logs, datasets, graders, replayable run histories
Deployment controller	Package and deploy the same harness to local dev, self-hosted, or managed cloud targets

A strong default IR should be **typed, event-capable, and runtime-agnostic**. One viable schema is:

```
{
  "template_id": "support-agent-v1",
  "mode": "graph",
  "agents": [
    {
      "id": "triage",
      "role": "root",
      "model_policy": "balanced-default",
      "instructions_ref": "prompt://support/triage",
      "allowed_tools": ["kb_search", "crm_lookup", "ticket_update"],
      "memory_policy": "thread-plus-user",
      "handoff_targets": ["billing", "technical"]
    }
  ],
  "tools": [
    {
      "id": "kb_search",
      "type": "mcp",
      "schema": "tool://kb/search",
      "approval_mode": "none",
      "timeout_ms": 4000
    },
    {
      "id": "ticket_update",
      "type": "http",
      "schema": "tool://ticket/update",
      "approval_mode": "required",
      "side_effect_level": "high"
    }
  ],
  "workflow": {
    "nodes": ["ingress", "retrieve", "plan", "act", "verify", "respond"],
    "edges": [
      ["ingress", "retrieve"],
      ["retrieve", "plan"],
      ["plan", "act"],
      ["act", "verify"],

```

```

        ["verify", "respond"]
    ],
    "checkpoint_policy": "async"
},
"context_policy": {
    "max_context_tokens": 120000,
    "compaction_strategy": "summary-plus-artifacts",
    "artifact_refs": true
},
"eval_suite": {
    "dataset_id": "support-golden-v3",
    "graders": ["resolution_correctness", "policy_compliance", "tone"],
    "blocking_thresholds": {
        "resolution_correctness": 0.92,
        "policy_compliance": 0.99
    }
},
"deployment": {
    "target": "managed-or-selfhosted",
    "streaming": true,
    "otel_enabled": true
}
}

```

That schema should compile into typed runtime contracts. A minimal interface set could look like this:

```

class ModelAdapter(Protocol):
    async def generate(self, messages, tools, response_schema, session) -> "ModelStep": ...

class ToolAdapter(Protocol):
    async def invoke(self, tool_name, arguments, run_ctx) -> "ToolResult": ...

class PolicyEngine(Protocol):
    async def pre_tool(self, proposed_call, run_ctx) -> "ApprovalDecision": ...
    async def post_output(self, output, run_ctx) -> "ValidationResult": ...

class StateStore(Protocol):
    async def load(self, session_id) -> "SessionState": ...
    async def checkpoint(self, session_id, state, step_event) -> None: ...

class Evaluator(Protocol):
    async def run_suite(self, run_id, dataset_id) -> "EvalReport": ...

```

```
class DeploymentTarget(Protocol):
    async def deploy(self, compiled_bundle, env) -> "DeploymentInfo": ...
```

A creator tool should also expose a **client-friendly streaming API**, not just REST polling. OpenAI's Codex App Server is a strong reference point here because it exposes the harness through a stable, bidirectional JSON-RPC surface that maps client requests into thread-managed core sessions and returns rich event updates for UI construction. That is a better model for a creator tool than a single `POST /run` endpoint with an opaque final answer. ⁶⁶

A concise HTTP surface could still complement streaming:

```
POST /v1/templates
GET /v1/templates/{id}
POST /v1/templates/{id}:compile
POST /v1/runs
GET /v1/runs/{id}
GET /v1/runs/{id}/events
POST /v1/runs/{id}/approvals
POST /v1/evals/runs
GET /v1/evals/runs/{id}
POST /v1/deployments
POST /v1/tools/mcp/register
POST /v1/tools/http/register
```

The most important design choice is to make **specs explicit and small**. OpenAI's harness-engineering guidance argues against giant monolithic instruction files and toward concise navigational context with structured docs close to code; Anthropic's long-running harness guidance and Microsoft's compaction guidance both point the same way. A creator tool should therefore generate a short root instruction file plus typed linked artifacts, rather than a single giant prompt blob. ⁶⁷

Implementation blueprint

At implementation time, the most important internal choice is to separate **append-only event history** from **materialized session state**. The event log is what you replay, audit, evaluate, and stream to the UI. The materialized state is what you use for fast resumption. This aligns with how modern harnesses expose sessions, traces, and checkpoints. OpenAI's tracing and session model, Anthropic's persistent event history, LangGraph's checkpoints, MAF's checkpoint/time-travel model, and AgentCore observability all point toward this split. ⁶⁸

For **logging and telemetry**, OpenTelemetry should be the default wire format. It is vendor-neutral, widely adopted, and already supported directly or indirectly by many leading harness stacks: OpenAI Agents, MAF, Foundry, Haystack, LlamaIndex, and AgentCore all expose or integrate with OTel-style tracing. A creator tool should therefore emit spans for at least `run`, `step`, `model_call`, `retrieval`, `tool_call`, `approval`, `checkpoint`, `compaction`, `evaluator`, and `deployment`. It should also attach standardized dimensions such as `session_id`, `template_id`, `agent_id`, `tool_name`, `tenant_id`,

`cost_usd_estimate`, `token_in`, `token_out`, `latency_ms`, `error_class`, and `policy_decision`. ⁶⁹

For **testing and evaluation**, the right strategy is layered.

Test layer	What it should prove	Practical implementation
Unit tests	Pure logic is correct	Schema validation, graph compilation, policy branching, pricing estimation
Contract tests	Adapters honor tool and model schemas	MCP contracts, function-call JSON schema, auth and retry behavior
Replay tests	Runtime changes do not break known runs	Deterministic replays over recorded traces and tool fixtures
Golden dataset tests	Business outcomes stay acceptable	Route fixed datasets through agents and compare graders/metrics
Trace grading	Trajectory quality, not just final answer	Grade tool choices, handoffs, approvals, safety checks, compaction quality
Load and soak tests	Scaling and durability are safe	Concurrent sessions, long-lived runs, resume-after-failure, storage pressure
Security tests	Safety controls actually work	Prompt injection, tool escape attempts, policy bypass, PII exfiltration, sandbox abuse
Canary tests	New runtime version is safe in production	Shadow traffic, subset routing, rollback on eval or latency regressions

This test stack is strongly supported by the landscape: OpenAI emphasizes datasets, trace grading, and Evals API; Foundry has built-in agent evaluators and tracing; ADK explicitly teaches golden datasets for regression detection; Haystack, LlamaIndex, CrewAI, and DSPy all expose evaluation or testing surfaces.

⁷⁰

For **CI/CD**, a strong default pipeline is: schema linting and typecheck, tool-contract validation, deterministic replay suite, offline eval suite, dependency/vulnerability scan, performance smoke test, then deployment to a canary environment with live tracing gates. DSPy's insistence on metric clarity, OpenAI's trace-based evaluation surfaces, and the platform-native observability stacks all argue that releases should be blocked on **measured quality**, not only code correctness. ⁷¹

For **deployment**, the creator tool should support three profiles because the user did not specify platform, budget, or SLA goals. A **local/self-hosted profile** should target containers and Kubernetes for teams that want data locality and custom infrastructure. A **managed-cloud profile** should target Agent Engine, Foundry, OpenAI-hosted surfaces, Claude Managed Agents, or AgentCore for fast productionization. A **hybrid profile** should keep the control plane centralized while letting tool/data execution live inside the customer VPC or private environment. That matches the market's actual operating modes. ⁷²

The recommended **default design pattern** for new harnesses is therefore:

A **single-agent or tightly bounded workflow baseline**, explicit state and checkpoints, typed tools behind a policy gateway, a short and structured instruction surface, eval-backed iteration, and upgrade paths into centralized multi-agent topologies only when task decomposition is demonstrably helpful. That is the pattern most consistent with Anthropic’s guidance, Google’s scaling results, and the architecture choices of the best current frameworks. ⁷³

Open questions and limitations

The main limitation in this landscape is **benchmark comparability**. Most public “framework benchmarks” are really architecture or model benchmarks, and often they are vendor-authored. The strongest fairness comes from benchmark harnesses like HELM and `lm-evaluation-harness`, but those mostly evaluate models and prompting/task setups, not the full operational harness layer. As a result, any claim that one runtime harness is universally best-performing should be treated skeptically. ⁷⁴

A second limitation is that **commercial popularity data is sparse**. Open-source stars are visible; enterprise adoption counts usually are not. In this report, commercial popularity metrics were therefore treated as ecosystem proxies where necessary, using linked open-source repos or official vendor usage signals such as ADK downloads and CrewAI’s certified-developer claim. ⁷⁵

A third limitation is that **this market is still moving quickly**. MCP is under active development, A2A is still young, Microsoft is in the middle of the Semantic Kernel to Agent Framework transition, and the meaning of “harness” itself is becoming more explicit only recently in AWS, Microsoft, Anthropic, and OpenAI materials. A new harness-creator tool should therefore be built around **adapters, typed IRs, and protocol abstraction**, not assumptions that today’s provider surfaces will remain stable. ⁷⁶

¹ ⁶³ <https://devblogs.microsoft.com/agent-framework/agent-harness-in-agent-framework/>
<https://devblogs.microsoft.com/agent-framework/agent-harness-in-agent-framework/>

² ⁵ ¹⁰ ¹¹ ¹² ⁶⁵ <https://github.com/langchain-ai/langgraph/tree/main>
<https://github.com/langchain-ai/langgraph/tree/main>

³ ⁶¹ ⁷³ <https://www.anthropic.com/research/building-effective-agents>
<https://www.anthropic.com/research/building-effective-agents>

⁴ ⁵⁵ ⁶² <https://docs.langchain.com/oss/python/langgraph/durable-execution>
<https://docs.langchain.com/oss/python/langgraph/durable-execution>

⁶ ⁴⁰ <https://developers.openai.com/api/docs/guides/migrate-to-responses>
<https://developers.openai.com/api/docs/guides/migrate-to-responses>

⁷ ⁵⁴ ⁷⁴ <https://crfm.stanford.edu/helm/>
<https://crfm.stanford.edu/helm/>

⁸ ⁹ <https://github.com/langchain-ai/langchain/blob/master/README.md?plain=1>
<https://github.com/langchain-ai/langchain/blob/master/README.md?plain=1>

¹³ <https://www.langchain.com/blog/benchmarking-multi-agent-architectures>
<https://www.langchain.com/blog/benchmarking-multi-agent-architectures>

14 17 <https://arxiv.org/abs/2308.08155>

<https://arxiv.org/abs/2308.08155>

15 <https://github.com/microsoft/autogen/tree/main>

<https://github.com/microsoft/autogen/tree/main>

16 <https://github.com/microsoft/autogen/releases>

<https://github.com/microsoft/autogen/releases>

18 56 <https://developers.llamaindex.ai/typescript/framework/tutorials/workflows/>

<https://developers.llamaindex.ai/typescript/framework/tutorials/workflows/>

19 20 https://github.com/run-llama/llama_index/tree/main

https://github.com/run-llama/llama_index/tree/main

21 57 <https://docs.haystack.deepset.ai/docs/components>

<https://docs.haystack.deepset.ai/docs/components>

22 <https://github.com/deepset-ai/haystack/tree/main>

<https://github.com/deepset-ai/haystack/tree/main>

23 <https://docs.crewai.com/en>

<https://docs.crewai.com/en>

24 53 <https://crewai.com/pricing>

<https://crewai.com/pricing>

25 52 <https://github.com/crewAIInc/crewAI/blob/main/README.md>

<https://github.com/crewAIInc/crewAI/blob/main/README.md>

26 58 <https://dsp.ai/>

<https://dsp.ai/>

27 28 <https://github.com/stanfordnlp/dspy/tree/main>

<https://github.com/stanfordnlp/dspy/tree/main>

29 <https://arxiv.org/abs/2406.11695>

<https://arxiv.org/abs/2406.11695>

30 31 32 <https://github.com/openai/openai-agents-python/tree/main>

<https://github.com/openai/openai-agents-python/tree/main>

33 34 35 <https://github.com/microsoft/agent-framework>

<https://github.com/microsoft/agent-framework>

36 <https://learn.microsoft.com/en-us/agent-framework/migration-guide/from-semantic-kernel/>

<https://learn.microsoft.com/en-us/agent-framework/migration-guide/from-semantic-kernel/>

37 38 39 <https://github.com/google/adk-python/tree/main>

<https://github.com/google/adk-python/tree/main>

41 <https://openai.com/api/pricing/>

<https://openai.com/api/pricing/>

42 <https://platform.claude.com/docs/en/home>

<https://platform.claude.com/docs/en/home>

- 43 <https://platform.claude.com/docs/en/about-claude/pricing>
<https://platform.claude.com/docs/en/about-claude/pricing>
- 44 <https://aws.amazon.com/bedrock/agents/>
<https://aws.amazon.com/bedrock/agents/>
- 45 <https://aws.amazon.com/bedrock/pricing/>
<https://aws.amazon.com/bedrock/pricing/>
- 46 <https://docs.aws.amazon.com/bedrock-agentcore/>
<https://docs.aws.amazon.com/bedrock-agentcore/>
- 47 <https://azure.microsoft.com/en-us/products/ai-foundry/agent-service/>
<https://azure.microsoft.com/en-us/products/ai-foundry/agent-service/>
- 48 <https://azure.microsoft.com/en-us/pricing/details/foundry-agent-service/>
<https://azure.microsoft.com/en-us/pricing/details/foundry-agent-service/>
- 49 <https://docs.cloud.google.com/vertex-ai/docs/start/introduction-unified-platform>
<https://docs.cloud.google.com/vertex-ai/docs/start/introduction-unified-platform>
- 50 <https://cloud.google.com/products/gemini-enterprise-agent-platform/pricing>
<https://cloud.google.com/products/gemini-enterprise-agent-platform/pricing>
- 51 <https://docs.langchain.com/langsmith/deployment>
<https://docs.langchain.com/langsmith/deployment>
- 59 <https://platform.claude.com/docs/en/managed-agents/overview>
<https://platform.claude.com/docs/en/managed-agents/overview>
- 60 <https://developers.openai.com/api/docs/guides/agents>
<https://developers.openai.com/api/docs/guides/agents>
- 64 <https://developers.openai.com/api/docs/pricing>
<https://developers.openai.com/api/docs/pricing>
- 66 <https://openai.com/index/unlocking-the-codex-harness/>
<https://openai.com/index/unlocking-the-codex-harness/>
- 67 <https://openai.com/index/harness-engineering/>
<https://openai.com/index/harness-engineering/>
- 68 <https://developers.openai.com/api/docs/guides/agents/integrations-observability>
<https://developers.openai.com/api/docs/guides/agents/integrations-observability>
- 69 <https://opentelemetry.io/docs/>
<https://opentelemetry.io/docs/>
- 70 <https://developers.openai.com/api/docs/guides/evals>
<https://developers.openai.com/api/docs/guides/evals>
- 71 <https://github.com/stanfordnlp/dspy/blob/main/docs/docs/learn/evaluation/overview.md>
<https://github.com/stanfordnlp/dspy/blob/main/docs/docs/learn/evaluation/overview.md>
- 72 <https://docs.cloud.google.com/run/docs/ai/build-and-deploy-ai-agents/deploy-adk-agent>
<https://docs.cloud.google.com/run/docs/ai/build-and-deploy-ai-agents/deploy-adk-agent>

⁷⁵ <https://cloud.google.com/blog/products/ai-machine-learning/more-ways-to-build-and-scale-ai-agents-with-vertex-ai-agent-builder/>

<https://cloud.google.com/blog/products/ai-machine-learning/more-ways-to-build-and-scale-ai-agents-with-vertex-ai-agent-builder/>

⁷⁶ <https://modelcontextprotocol.io/specification/2025-03-26>

<https://modelcontextprotocol.io/specification/2025-03-26>